

◆ 1. What You're Trying to Capture

When multiple sessions try to lock the same resource in conflicting ways, SQL Server detects a **deadlock** — it picks one transaction as the *victim* and rolls it back.

We want to **capture** these events so we can:

- See **which queries** were involved
 - Identify **which resources** were locked
 - Find **which session** was chosen as the victim
-

◆ 2. Create a Deadlock-Capture Extended Event Session

You can do this from SSMS (graphically) or via T-SQL.

Let's use T-SQL for clarity 🙋

-- Create an Extended Events session for deadlocks

```
CREATE EVENT SESSION [Track_Deadlocks]
```

```
ON SERVER
```

```
ADD EVENT sqlserver.deadlock_graph
```

```
ADD TARGET package0.event_file
```

```
(
```

```
    SET filename = 'C:\XE\Deadlock_Capture.xel',
```

```
    max_file_size = 10,
```

```
    max_rollover_files = 5
```

```
)
```

```
WITH
```

```
(
```

```
    MAX_MEMORY = 4MB,
```

```
    EVENT_RETENTION_MODE = ALLOW_SINGLE_EVENT_LOSS,
```

```
    MAX_DISPATCH_LATENCY = 30 SECONDS,
```

```
TRACK_CAUSALITY = ON,  
STARTUP_STATE = ON  
);  
GO
```

-- Start the session

```
ALTER EVENT SESSION [Track_Deadlocks] ON SERVER STATE = START;  
GO
```

Explanation:

- sqlserver.deadlock_graph → Captures XML deadlock graphs.
- event_file → Saves the event output as .xel files.
- STARTUP_STATE = ON → Ensures it runs automatically if SQL restarts.

◆ **3. Simulate a Deadlock (Optional)**

Here's a small test to generate one for learning purposes (run in two SSMS windows):

Session 1:

```
BEGIN TRAN;  
  
UPDATE Orders SET Amount = Amount + 1 WHERE OrderID = 1;  
  
WAITFOR DELAY '00:00:10';  
  
UPDATE Customers SET Name = 'ABC' WHERE CustomerID = 1;  
  
COMMIT;
```

Session 2:

```
BEGIN TRAN;  
  
UPDATE Customers SET Name = 'XYZ' WHERE CustomerID = 1;  
  
WAITFOR DELAY '00:00:10';  
  
UPDATE Orders SET Amount = Amount + 1 WHERE OrderID = 1;
```

COMMIT;

➡ One of them will get a **deadlock error**, e.g.:

Transaction (Process ID 54) was deadlocked on resources with another process...

◆ 4. Analyze Captured Deadlocks

Now view your captured events.

Option 1: From SSMS GUI

1. Go to:
2. SSMS → Management → Extended Events → Sessions → Track_Deadlocks → Watch Live Data
3. You'll see a **deadlock event**.
4. Right-click → **“View Target Data”** or **“Deadlock Graph”**.

You'll get a **visual deadlock diagram** showing:

- Which query locked which resource
 - Which session was chosen as victim
 - Lock types (shared/exclusive)
-

Option 2: Read from file manually

You can query the event file using T-SQL:

SELECT

event_data.value('(event/@name)[1]', 'nvarchar(50)') AS EventName,

event_data.value('(event/data/value)[1]', 'varchar(max)') AS DeadlockGraph

FROM

sys.fn_xe_file_target_read_file('C:\XE\Deadlock_Capture*.xel', NULL, NULL, NULL)

CROSS APPLY

(SELECT CAST(event_data AS XML) AS event_data) AS X;

You can open the XML graph in SSMS or any XML viewer for details.

◆ 5. Stop or Remove the Session (when done)

```
ALTER EVENT SESSION [Track_Deadlocks] ON SERVER STATE = STOP;  
DROP EVENT SESSION [Track_Deadlocks] ON SERVER;
```

◆ 6. Extended Events for Blocking Detection

To capture blocking (long waits), you can also add events like:

```
CREATE EVENT SESSION [Track_Blocking]  
ON SERVER  
ADD EVENT sqlserver.lock_acquired,  
ADD EVENT sqlserver.lock_released,  
ADD EVENT sqlserver.blocked_process_report  
(  
    ACTION(sqlserver.sql_text, sqlserver.session_id)  
)  
ADD TARGET package0.event_file (SET filename = 'C:\XE\Blocking.xel');  
GO
```

```
ALTER EVENT SESSION [Track_Blocking] ON SERVER STATE = START;
```

➡ You must enable the blocked process report threshold:

```
EXEC sp_configure 'show advanced options', 1;  
RECONFIGURE;  
EXEC sp_configure 'blocked process threshold', 10; -- in seconds  
RECONFIGURE;
```

This means: “Report any blocking lasting more than 10 seconds.”

✖ Summary Table

Task	Event	Description
Detect Deadlocks	sqlserver.deadlock_graph	Captures XML deadlock graph
Detect Blocking	sqlserver.blocked_process_report	Captures blocking longer than N seconds
Detect Long Queries	sqlserver.rpc_completed	Captures slow stored procedures
Detect Waits	sqlserver.wait_info	Monitors waits (CPU, IO, etc.)

✔ Why Extended Events Are Best

- **Low overhead** (unlike Profiler)
- **Run continuously**
- **Store structured event data**
- **Can auto-start on SQL restart**